

Acting for an account

An exploration of the ways several people could act as one atproto account, without sharing its secret. A community, a brand, a newsroom, a band: one DID, one repo, and moderators or staff who legitimately write to it.

Where things stand

A shared password, or app passwords handed around. Every write looks the same on the wire. Nobody can be removed without rotating the secret. Nothing records who did what.

The options

Several ways to do this have been proposed. Each is set out on the same axes, with a diagram, what it keeps, and what it costs.

What discussion has surfaced

Ideas

- **The app holds the account's session.** The community signs in to the app once; the app decides which of its users may use that session.
- **Delegated scopes at the person's own server.** A person's authorization server grants an app the right to act for other accounts, as new scopes.
- **Sign in as the account.** The account's server authenticates the person at *their* server, checks their standing, and issues the account's session, narrowed.
- **A group host.** A dedicated service that is the PDS, the authorization server, and the roles for community accounts, with an admin API for apps.
- **Configuration on the account.** The account names who may act for it; its own PDS enforces that on writes, on sign-in, and at creation.

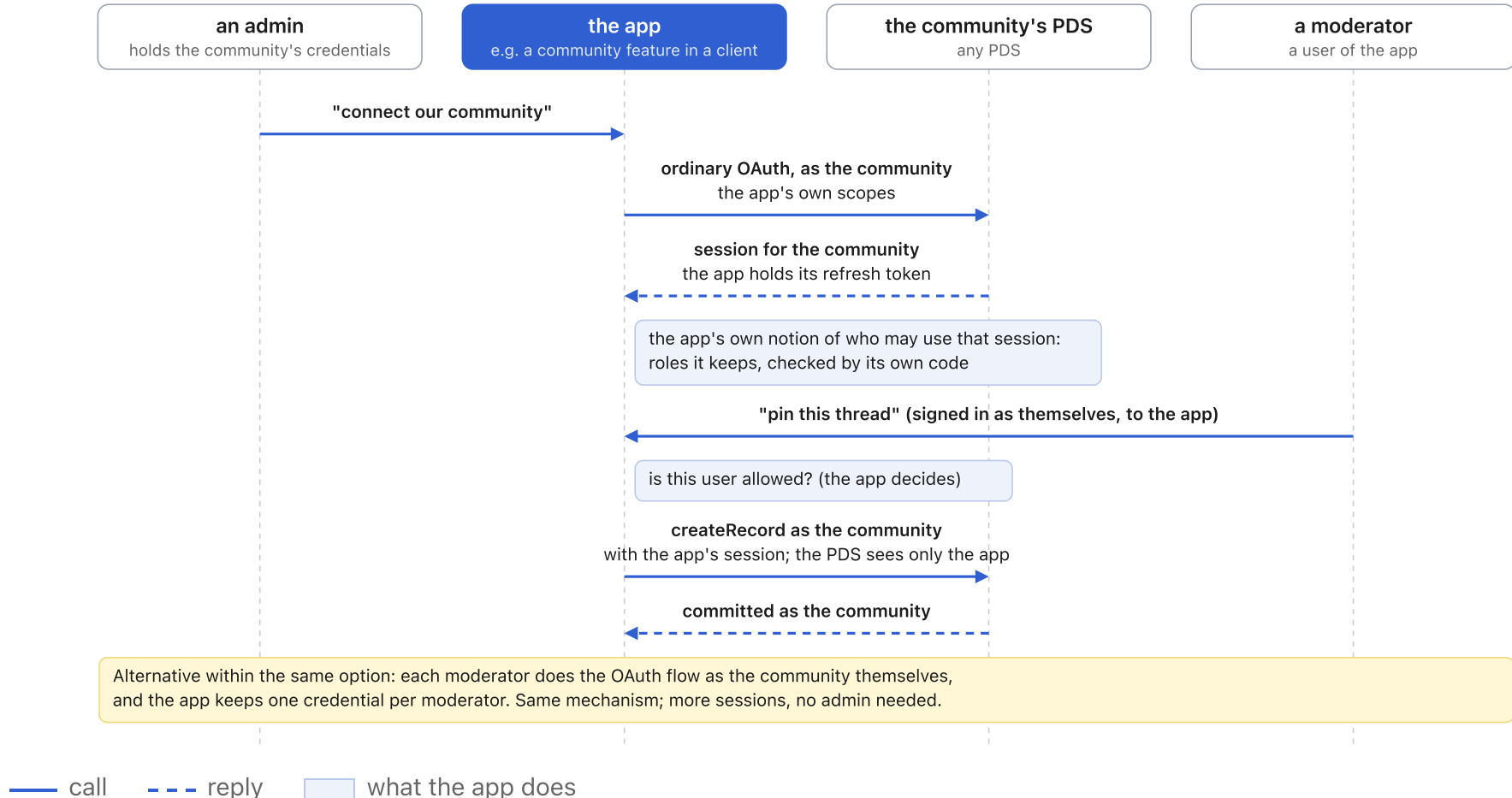
Questions and concerns

- **Consent has to be legible.** What does the person see, and does it name an audience? An open audience is a different decision for five friends than for a community of thousands.
- **Delegate writes, not everything.** Identity, account, and general calls are the dangerous parts; repo, space, and blob writes are the need.
- **Protocol changes are expensive.** A new delegated-auth mechanism is months of work; a chain-of-delegation token format may be the long-term answer.
- **Existing clients should keep working.** A moderator should not need a special app.
- **Standing should travel with the person,** not be locked inside one app.
- **Who holds the credential?** An app, a host, or nobody.
- **Attribution and revocation.** Who did this, and how fast can they be stopped.
- **Portability.** Can a community move, and does its arrangement move with it?

The axes

Where does the policy live?	In the app · in the authorization server of the account · in a dedicated host · on the account, as host state any PDS can read
Who authenticates, and where?	Nobody in particular · the person at their own PDS · the person at the account's server
What does the client hold?	The account's session · the person's session · a narrowed session for the account
Where is consent shown, and to whom?	At the person's server (an audience question) · at the account's server, per account · nowhere
What can be delegated?	Everything · presets · writes only
Attribution and revocation	On the wire, in a log, inside one app, or not at all · immediate, next request, or rotate a secret
Can the account live anywhere, and move?	Any PDS · only a server that implements the mechanism · only a dedicated host
What has to change?	Nothing · the app · a server implementation · the reference PDS · the protocol

A · The app holds the account's session



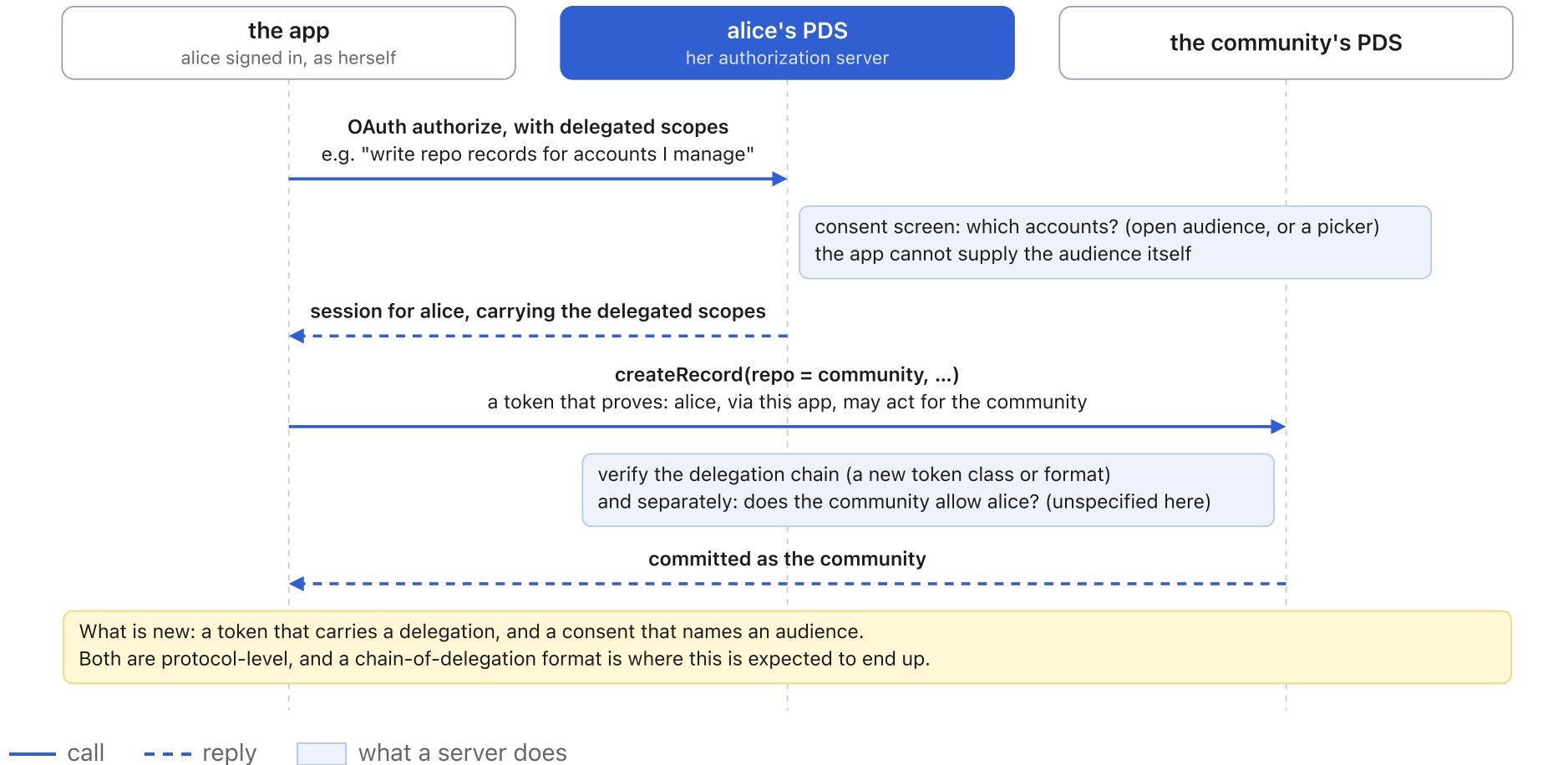
Keeps

Nothing changes anywhere. Plain OAuth, one consent, honest about who granted what to whom: the account authorized the app. Works today with any PDS.

Costs

The app holds a credential for the account, with the app's scopes, not any one person's. Roles live inside the app and do not travel. Attribution exists only in the app's own records. Each app must be authorized separately by someone with the account's credentials.

B · Delegated scopes at the person's own server



Keeps

Standing travels with the person: one login, one session, and the app acts for whichever accounts allow it. A simple consent screen if the audience is left open.

Costs

A new protocol mechanism: a token that carries a delegation the account's PDS must verify, which is the kind of change that takes months and several implementations. Consent needs an audience picker the app cannot fill in. The account's own side, who allowed this person, still has to live somewhere.

C · Sign in as the account, at its own server



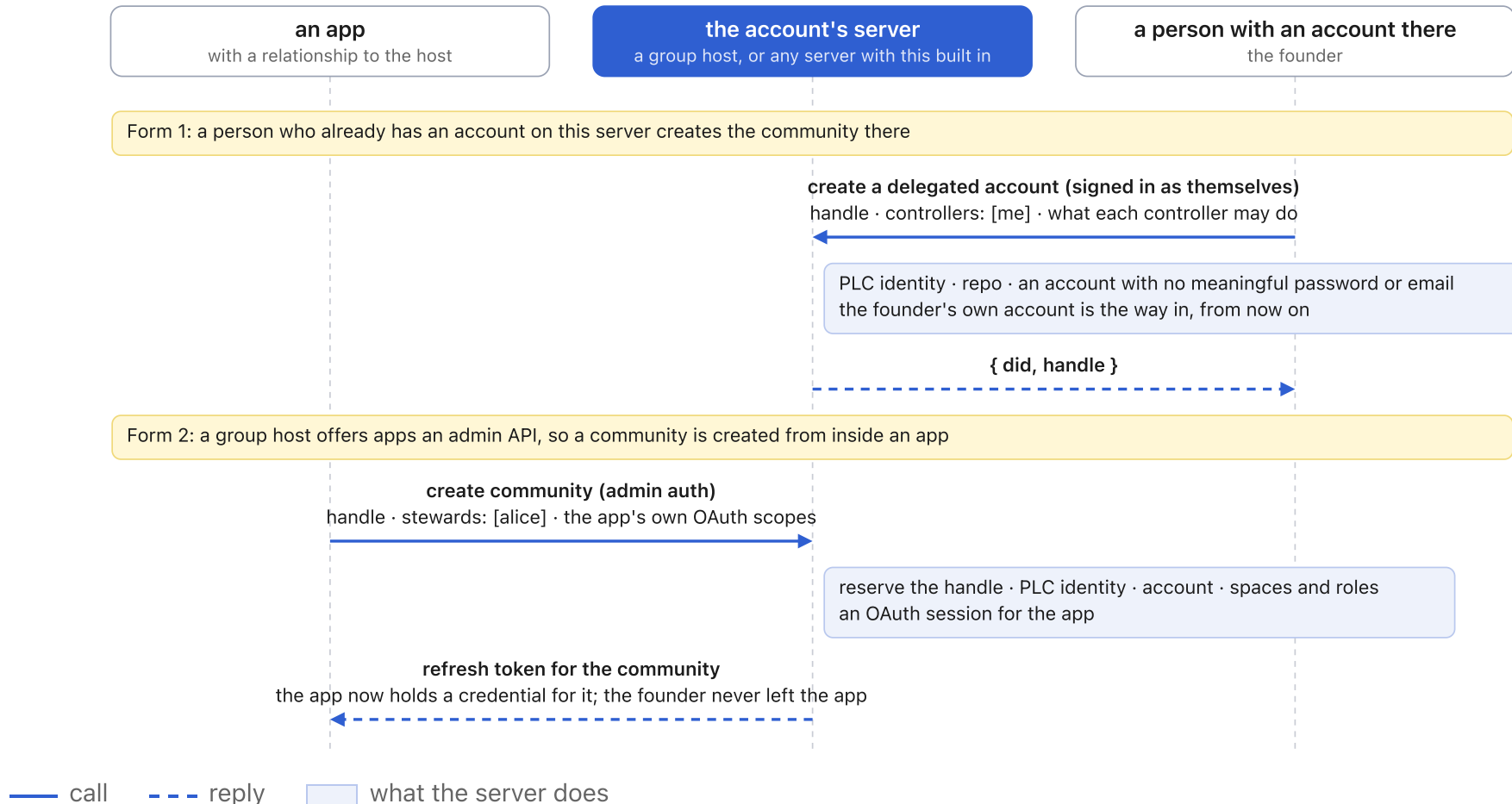
Keeps

Every existing OAuth client works, because it only sees an ordinary login. Consent is per account, at the account's server, with the client's own scopes. No protocol work. Runs today wherever the server implements it.

Costs

The account must live on a server that implements the mechanism. Roles, the login method, the narrowing, and the log are that server's own, with no shared representation, so moving the account leaves its arrangement behind.

C · How the account comes to exist



Keeps

Two forms, both without protocol work. A person who already has an account on that server creates the community there and names its controllers. Or a dedicated group host offers apps an admin API, so a community is created from inside an app without the founder leaving it.

Costs

The founder must have an account on that server, or the app must have a relationship with that host. In the admin-API form the app receives and keeps a session for the community, with the app's scopes rather than any one person's. Either way the community is born on that server and stays there.

D · Configuration on the account

Host state on whatever PDS the account lives on. Like app passwords or preferences: not in the repo, not on the firehose.

controllers	DIDs that may manage this configuration. Not delegates.
delegates	DIDs that may write, each with permissions in auth-scope syntax: <code>repo: , space: , blob: only</code> . Never <code>rpc: , identity: , account: .</code>
policy	delegate-list: the list above. managing-app: ask a named service (a community host) <code>checkDelegate(account, did)</code> , signed as the account, and cache its answer.



D1 · Delegated write

The person's own app, their own session. A service auth token from their PDS, `repo` set to the account. One login; the client must know how to address another account.



D2 · Sign in as the account

Option C's flow, with the policy read from the configuration instead of hooks. Any OAuth client, unchanged; the session narrowed, with `act.sub` .

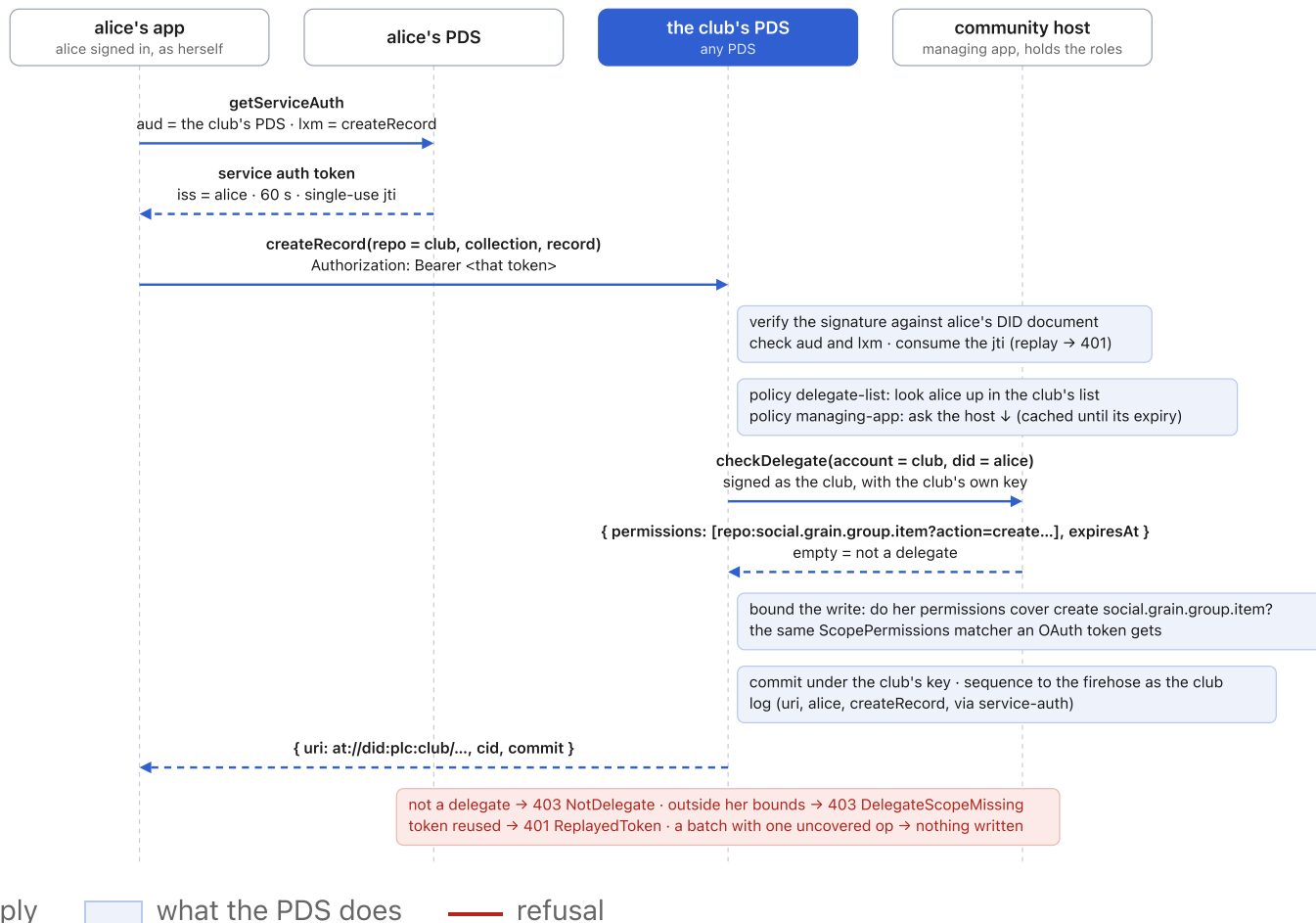


D3 · Create from an app

The founder's app creates the account on any PDS with the founder's own service auth. No usable credentials; the founder is a controller. The app holds nothing.

Same bounds, same matcher the PDS already uses for OAuth scopes, same per-write log for all three.

D1 · Delegated write



The app needs one thing from the person's PDS: an `rpc:` permission to mint service auth for the account's PDS. Naming that PDS as the audience keeps the consent bounded; a general-purpose client has to ask for an open audience, and the consent should say so in words.

D2 · Sign in as the account



The same shape as option C. The difference is where the answer to "may this person act, and how far" comes from: the account's configuration, which any PDS could read, rather than the server's own hooks and roles.

D2 · What a narrowed session gets

The client asked for

```
atproto
repo:social.grain.group.item
repo:app.bsky.feed.post?action=create
blob:image/*
```

The account allows alice

```
repo:social.grain.group.item?
action=create&action=delete
```

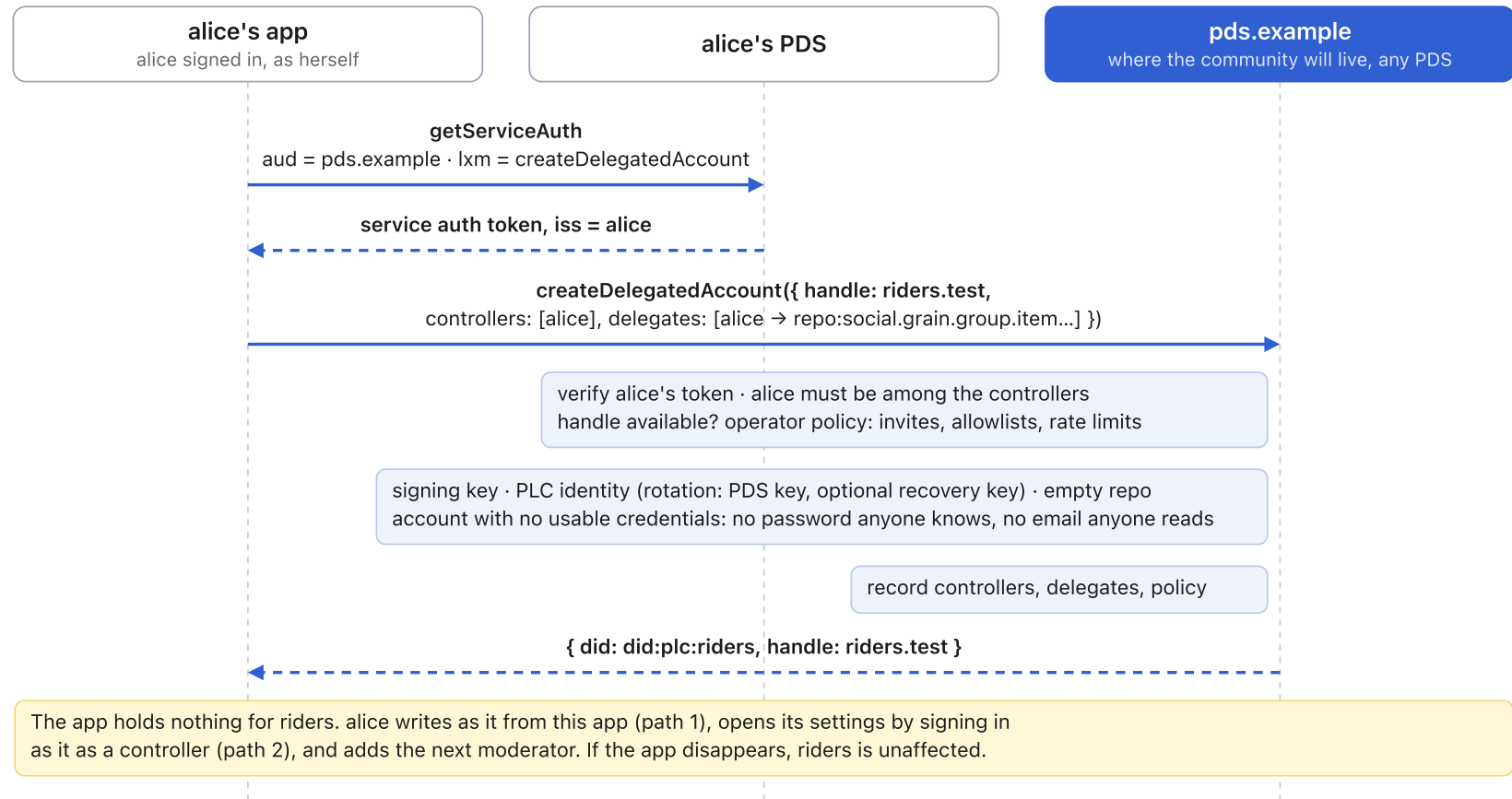
The token alice gets, as the account

```
atproto
repo:social.grain.group.item?
action=create&action=delete
repo:app.bsky.feed.post?action=create
blob:image/*
act: { sub: did:plc:alice }
```

The intersection, applied at issue and again on every request against the account's *current* configuration. Remove alice and the session is refused on its next use and cannot be refreshed.

A **controller** additionally keeps `account:delegates?action=manage`, so the account's settings tool works for them. Nobody else's session keeps any `account:`, `identity:`, or `rpc:` scope.

D3 · Create a community from an app



— call - - - reply what the PDS does

Same steps as `createAccount`, minus credentials anyone could use. Operator policy (invites, allowlists, rate limits) applies as it does there. An account with no credentials of its own must keep at least one controller.

D · Who may do what

	the account's own password if it has one	a controller	a delegate	anyone else
write as the account	yes	only if also a delegate	yes, within their permissions	NotDelegate
sign in to a client as the account	yes, full scope	yes: writes per their delegate entry + account:delegates	yes: writes only, narrowed	refused at sign-in
manage delegates and controllers	yes	yes: signed in as the account, or by service auth from their own session	NotController	no
create the account	n/a	the first controller, by service auth	no	no
reach identity, email, password, keys	yes	never	never	never
be removed, effect	rotate	next request	next write, or next request of a session	

Keeps

The account can live on any PDS and move with its arrangement. One session or any client, the person's choice. Writes only. Attribution on the wire and in a log; revocation on the next request. No credential held by any app or host.

Costs

The reference PDS changes: a configuration, one auth path on writes, one login method, one claim, and a creation method. It is not deployable by one operator alone if the goal is "any PDS". For general-purpose clients on D1, the consent at the person's server is still an open audience.

Side by side

	shared password / app passwords	A · app holds the session	B · delegated scopes at the person's server	C · sign in as, at the account's server	D · configuration on the account
where the policy lives	nowhere	in the app	the person's AS grants; the account's side unspecified	the account's AS, or a group host	the account's config; roles optionally in a managing app
who authenticates	nobody in particular	the account, once	the person, at their PDS	the person, at their PDS, via the account's AS	the person, at their PDS
the account can live on any PDS	yes	yes	yes	no	yes
existing OAuth clients work unchanged	n/a	the one app	no	yes	yes (D2); D1 needs the client to know
consent shown where	none	account's AS, once, to the app	person's AS; audience open or picked	account's AS, per account	account's PDS per account (D2); person's PDS, audience-specific or open (D1)
what may be delegated	everything	the app's scopes	as proposed: repo, space, blob; others debated	presets, up to everything	writes only
attribution	none	in the app	in the token chain	act , host log	act + a per-write log
revocation	rotate	the app's, or revoke the app	revoke the grant	revoke sessions	next request
arrangement moves with the account	n/a	no	with the person	no	yes, as host state
how the account is created	ordinary createAccount; someone keeps the password	the same; then an admin connects it to the app	the same; unspecified beyond that	by a person with an account on that server, or by an app through a host's admin API	by the founder's own service auth, on any PDS; no credentials, controllers instead
what changes	nothing	nothing	the protocol: a new token mechanism	a server implementation, or a new service	the reference PDS

Open questions

- **Discovery.** How a person's client learns which accounts they may act for, without every PDS notifying every other. Every option shares this.
- **The open audience.** Whether a general-purpose client's consent at the person's server can be made legible, or whether per-account consent at the account's server is the only honest form.
- **Calling out as the account.** Whether a session as the account should ever be able to call a named service as the account, and how narrowly.
- **Portability.** Whether the arrangement (who may act, and how far) should be a defined piece of host state that migrates, whichever option produces it.
- **Consent records.** Keeping a delegate's consent to a client apart from the account's own.
- **Operator policy on creation.** Who may create accounts this way, and how many.
- **The long view.** Whether a chain-of-delegation token format eventually replaces any of this, and what should be built in the meantime.